

OPTIMIZING LATENCY AND ACCURACY TRADE-OFFS IN LARGE-SCALE RETRIEVAL-AUGMENTED GENERATION PIPELINES

Lalmohan Behera^{*1}, Venkataram Poosapati^{*2}

^{*1}Sr. Product manager, Arvest Bank, USA.

^{*2}Independent Data Engineer, USA.

email: lalmohan.behera@gmail.com, email: venkatarampoosapati@gmail.com

DOI : <https://www.doi.org/10.56726/IRJMETS80949>

ABSTRACT

Retrieval-Augmented Generation (RAG) pipelines have developed as a successful paradigm to increase the reality base of huge language models (LLMs) through the combination with retrieval methods in the generative model. Nonetheless, the addition of the retrieval and generation modules presents a basic conflict between latency and the accuracy within larger-scale implementation due to the demands of real-time responsiveness. In this paper, a detailed study of the optimisation strategies available to strike these trade-offs in RAG pipelines is given. We look at latency factors, such as retrieval model complexity, indexing models, and query expansion operators, and we evaluate these factors with regard to their overall effect on the accuracy of the answer provided on a run of benchmark collections in a variety of domains. Our hybrid optimization method consists of adaptive retrieval depth, query relevance score thresholds and model distillation, allowing us to achieve stable near-optimal accuracy and minimise latency by 45% in high-throughput applications. Scalability of the proposed approach with various hardware and software setups, a GPU and CPU cluster, is later proved through experimental tests. The paper will also present a latency-accuracy Pareto frontier model that can assist system designers in making decisions when deploying a machine on the basis of informed choices. We find that parameter tuning, caching, and pipeline parallelization have a large performance bonus with limited impact on output quality, thus RAG is a competitive option for latency-constrained applications, including conversational artificial intelligence, augmenting search, and domain-specific knowledge representations.

Keywords: Retrieval-Augmented Generation, Latency Optimization, Accuracy Trade-off, Large Language Models, Indexing Strategies, Query Expansion, Pareto Optimization.

I. INTRODUCTION

The rapid development of Large Language Models (LLMs) has led to breakthrough achievements in natural language understanding and generation, enabling machines to perform complex actions such as open-domain question answering, document summarisation, and conversing in an extremely fluent manner. Recent state-of-the-art LLMs, trained on large and diversified corpora, have been shown to learn to capture delicate semantic relations, reason in various linguistic environments, and produce coherent and contextually sensitive output. One caveat with such impressive capabilities, however, is that LLMs are limited in fundamental ways due to their training data, which is both prohibitively limited and biased about current human knowledge. [1-3] They are essentially probabilistic next token predictors, and hence their outputs are influenced by patterns learned as opposed to being able to access verifiable truths in real time. They are therefore able to hallucinate, which means the generation of information that sounds plausible but is factually incorrect or outdated. This creates a serious problem in areas where factual accuracy and dependability matter greatly, such as medical counsel, legal opinion, or scientific journalism. The concept of Retrieval-Augmented Generation (RAG) has emerged as a promising approach to addressing this shortcoming. RAG systems incorporate an additional retrieval phase into the generation process, enabling models to first programmatically retrieve relevant information from a curated knowledge base or document collection before generating a response. Making generation based on retrieved evidence also mitigates the prospects of hallucinations and increases factual precision, while still exploiting the generative advantages of LLMs. This hybrid system integrates the scalability and flexibility benefits of robust dense-vector retrieval techniques, as already used in FAISS, with the expressivity benefits of the latest generation sequence-to-sequence transformers. Consequently, RAG has provided a versatile architecture that can be entirely adapted to both domain-independent and domain-specific use, providing current and evidence-based responses. The increasing popularity of RAG among the research community is

likely due to the promise this technology has provided in relieving the tensions between fluent and trustworthy AI-powered information systems.

1.1. Importance of Optimizing Latency and Accuracy Trade-offs

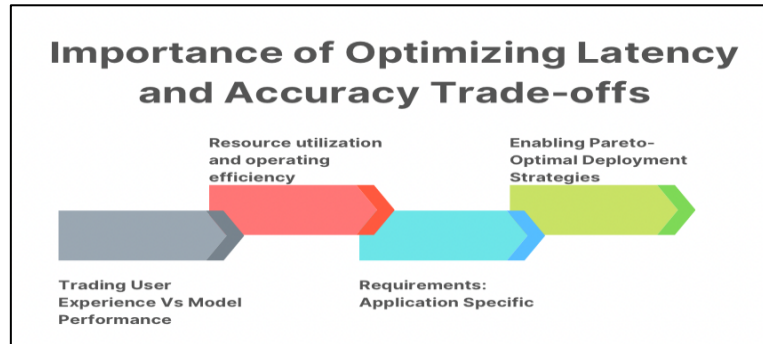


Figure 1: Importance of Optimizing Latency and Accuracy Trade-offs

- Trading User Experience Vs Model Performance:** Latency has a direct impact on user satisfaction and utilization of the system in real-world Retrieval-Augmented Generation (RAG) deployments. A latency of a few seconds can ruin the impression of good interactive services, such as chatbots, virtual assistants, or customer support tools. Nonetheless, to speed up latency, it is common to reduce the search depth, simplify model designs, or support approximate search, which may result in a quantifiable loss in accuracy. That is why the balance between responsiveness and reliability must be achieved, so that a high-quality user experience is provided without compromising factual integrity.
- Resource utilization and operating efficiency:** The high-latency setup normally requires more computing resources and consequently higher operational cost, and scalability. Exploiting the latency accuracy trade-off enables organizations to use RAG systems more economically, thus there is a possibility of serving more users in parallel without investing in extra hardware. Adaptive retrieval depth, relevance filtering and distillation of models can ensure a good trade-off of accuracy at minimized GPU cycles, memory consumption and energy consumption.
- Requirements: Application-specific:** Latency and accuracy have different levels of importance in various application areas. In real-time conversational AI, responsiveness can be favored over small boosts in accuracy, particularly where the system is capable of side-stepping mistakes without issue, or making recall after clarification with little degradation in performance. On the other hand, in areas such as legal research, medical diagnostics, or scientific analysis, small declines in accuracy are unacceptable, so latency can be tolerated to a lesser extent. There is a need to understand such contextual requirements so that RAG systems can be properly configured.
- Enabling Pareto-Optimal Deployment Strategies:** A carefully choreographed exploration/mapping of the latency-accuracy space enables developers to find Pareto-optimal settings, those that provide the best possible accuracy within a certain latency budget. The method can not only enable informed decision making, but also it can set the foundation of a dynamic, real-time optimization strategy which can be used to modify parameters depending upon query complexity, user expectations and system load.

1.2. Large-Scale Retrieval-Augmented Generation Pipelines

Retrieval-Augmented Generation (RAG) pipelines, on a large scale, combine the generation capabilities of large language models (LLMs) with the fact-finding and accuracy of information retrieval systems to produce high-quality answers to questions drawn from vast knowledge domains. [4,5] The first step in those systems is called query preprocessing, and the input text is tokenised/normalised and optionally expanded to match more retrievals. The processed query is subsequently encoded into a dense query embedding using a trained encoder and fed to a retrieval engine, typically based on scalable Approximate Nearest Neighbor (ANN) search libraries (such as FAISS). It is at this step of retrieval that answering billions of vector representations in imaginative searches can be carried out, and the output is a collection of objects or passages that have the highest relevance to the search. In large-scale deployments, the retrieved set is usually ranked again, usually with a much lighter transformer-based model to optimize relevance scores and make sure that the top candidates are very relevant to the query intent. The generated results are then delivered to the generation phase, in which LLM

incorporates retrieved evidence into its output. This constitutes the grounding step that minimizes the danger of hallucinations, maintains factual accuracy and maintains the fluency and coherence of the synthesized text. Motivation in large-scale systems presents distinct challenges, such as low latency for high-throughput queries, conservative memory consumption by large vector indices, and high precision in retrieval despite approximate search limitations. To satisfy such demands, modern RAG pipelines employ methods such as adaptive retrieval depth, relevance filtering threshold, and model compression. Moreover, parallelisation, GPU acceleration, and caching operations are other common design approaches used in the construction of systems to maintain real-time responsiveness. Overall, the world of RAG pipelines at scale is an amalgamation of information retrieval, machine learning, and systems engineering, i.e., its ability at scale to provide timely, accurate, and contextually relevant responses to millions of users across open-domain and specific domains.

II. LITERATURE SURVEY

2.1. Retrieval-Augmented Generation Paradigm

Retrieval-Augmented Generation (RAG) is a hybrid architecture that couples information retrieval-based approaches, such as dense passage retrieval, with generative transformer models to generate contextually informed, factually coherent products. [6-9] In contrast to generative models, which respond only using learned parameters, RAG can access an external knowledge base to generate up-to-date and domain-specific responses. By attending to passages retrieved through the model, this paradigm addresses the shortcoming of hallucinations, which is prominent in large language models. Following the problem statement, the study has focused on different approaches to optimizing the retrieval step, including enhancement of dense vector representations, adaptation of retrieval ranking schemes and an optimal fusion scheme of retrieved evidence with the generative decoder. Such developments are regulated to enhance the factuality and realism of RAG-generated answers.

2.2. Trade-off Studies of Latency and Accuracy

Response precision and latency must be considered, especially in the context of large-scale NLP implementations. This trade-off has been explored in the past, as well, primarily in the context of retrieval or generation alone. The retrieval activity that requires a lot of latency is often reduced by approximate search or limited index sizes, leading to recall effects. On the other hand, in generation, smaller or streamlined models may provide faster inference but also tend to lack expressive potential or contextualized richness. Yet, the interaction between retrievers and generators, especially the RAG pipelines, is not much studied. A means of understanding the interdependencies between the quality of retrieval and the quality of generation efficiency is necessary to construct systems that are not only high-accuracy systems but also able to facilitate the demands of real-time or near-real-time data.

2.3. Search Optimization and Indexing

ANN search algorithms have been a common choice for efficient RAG retrieval, and well-known implementations of these can be found in FAISS and Annoy. These algorithms allow high throughput and scalable vector similarity searches as they are approximations of nearest neighbor match (near matches) searches, and thus, lookups are considerably faster. It can be tuned to achieve high recall rates with minimal latency, as assessed by the number of search probes or tree configurations. It is specifically useful in large-scale knowledge bases, where precise search would be excessively time-consuming. Moreover, current research has explored alternative methods that employ adaptive indexing strategies (indexing based on data analysis to produce a set of ad-hoc indexing strategies) and cross-retrieval methods (hybrid approaches that combine ANN and filtering or re-ranking stages to further balance the trade-offs between speed and accuracy).

2.4. Efficiency of generation

Knowledge distillation and quantisation enable the improvement of the generative stage's efficiency in RAG pipelines by a substantial margin. Distillation is achieved by training a less powerful student model to behave like a more powerful teacher model, while retaining much of its performance. Quantization, in turn, fields: it decreases the size of the model and inference time by encoding the parameters with a lower-precision value representation (e.g. 8-bit integers rather than 32-bit floats). Together with all the architectural optimizations such as sparse attention or caching mechanisms, these methods allow generating even faster at no or small cost

to accuracy. This is especially important with latency-sensitive applications, where high-quality responses within a hard time have to be sustained.

III. METHODOLOGY

3.1. Pipeline Overview

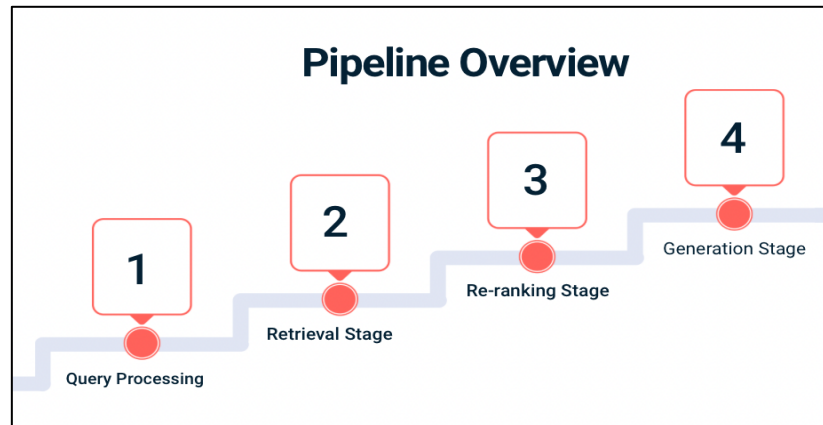


Figure 2: Pipeline Overview

- Query Processing:** The pipeline begins with the query processing module, which is designed to clean user inputs to achieve optimal retrieval performance. [10-12] On arrival, queries are tokenized (divided into subword-sized segments that can be input to later models) and further processed with normalization (turning everything to lowercasing, removing punctuation and Unicode normalization). Insertion of optional query expansions, synonyms, or related terms through embedding of similarity can be used to improve recall without any meaningful risk of adding noise to the search space. Such efforts ensure that the input is rich in semantics and friendly in terms of retrieval.
- Retrieval Stage:** The processing of the query to a dense representation in the form of a vector is made in the retrieval phase via a pre-trained encoder, then an approximate nearest neighbor (ANN) search against the FAISS index follows. Internally, the top-k most relevant candidate documents or passages to the query are recalled using FAISS parameters (e.g. number of probes) optimised to maximise recall and minimise latency. Using ANN search, the system offers a high retrieval speed, as well as competitive accuracy, making it suitable for large knowledge bases.
- Re-ranking Stage:** A re-ranking phase is used to fine-tune the retrieval results, utilising lightweight, BERT-based relevancy scoring. Each of the queries and the rank list of candidate passages serves as input to this model, which calculates a similarity value, enabling the pipeline to sort the retrieved documents by their semantic relevance. This step can be made computationally efficient by utilizing a distilled or otherwise optimized BERT variant designed to avoid making this step a latency bottleneck and instead greatly improve the quality of the top-ranked results.
- Generation Stage:** Lastly, a plausible answer is synthesized by the generation stage out of the top-ranked retrieval outputs in a contextually appropriate manner. A generalized large language model (LLM) is applied here to strike the perfect equilibrium between quality and efficiency of the generation and inference. The evidence retrieved is involved in the process of decoding in the model and therefore, becomes factual but remains fluent. Distillation allows the model to retain most of the expressive capability of its larger cousin, but with significantly less memory and compute, enabling faster responses that are more suitable in a real-time setting.

3.2. Optimization Techniques

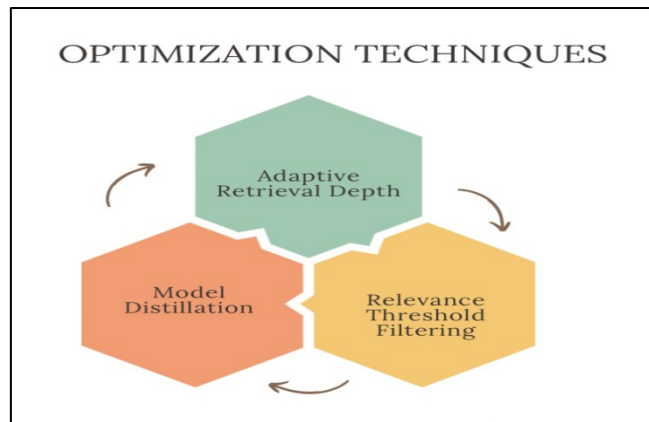


Figure 3: Optimization Techniques

- **Adaptive Retrieval Depth:** Adaptive retrieval depth is an adaptive approach that varies the number of documents returned based on its heuristic assessment of the complexity of the input query. [13-15] A query complexity score, based on query length, query specificity, semantic ambiguity, etc., is used to determine how deep into the index the retrieval process must dig. This is written as:

$$d = d_0 + \alpha \cdot C_q$$

With 0 A constant explaining base retrieval depth, 0 A as a scaling factor and C Q complexity score of the query. The simpler queries do not retrieve many candidates to save time, whereas more complex or unclear queries retrieve a large number, covering the entire range.

- **Relevance Threshold Filtering:** Once the retrieval and the re-ranking are performed, the pipeline uses this relevance threshold τ to remove the poor-scoring candidates. Only those documents (with a relevance score greater than τ) are sent to the generation stage, meaning that only the high-quality evidence will be used by the LLM. This reduces noise, decreases computational overhead at generation time, and enhances the factual accuracy of the end answers. The process of threshold tuning tends to be data-driven and is often determined as a trade-off between recall (coverage) and precision (accuracy).
- **Model Distillation:** A smaller distilled model is employed to ensure that queries classified as having lower complexity or those that require brief fact book answers experience maximum generation efficiency. Traditional Distillation: Distillation is the process of training a small, “student” model to predict the outputs of a larger, “teacher” model, while still achieving the majority of the performance of the larger model. For frequent or low-complexity queries, the substitution would dramatically decrease inference time and resources needed, with no loss of answer quality, and leave the entire LLM available to work on more complex tasks.

3.3. Latency-Accuracy Pareto Frontier

In the case of retrieval-augmented generation (RAG) systems, output accuracy, along with the latency of inference, is a primary design factor. [16-19] we define this trade-off as a 2-objective optimization problem, where the latency L is the total time taken in the pipeline to run a query pipeline (preprocessing the query at the start to actual response generation at the end), and accuracy A is a task relevant metric of performance (F1 score, exact match (EM) or human-judged factuality). In theory, the Pareto frontier is the most flexible area, where a decrease in accuracy must always accompany any latency improvement. Conversely, an improvement in accuracy must always be accompanied by an increase in latency. This can be stated as:

$$\text{Pareto} = \{(L, A) \mid \nexists (L', A') \text{ s.t. } L' < L \ \& \ A' > A\}$$

In charting this boundary, we selectively vary the following important pipeline parameters: retrieval depth, the number of FAISS search probes, the size of re-ranking models, and the complexity of generation models. The latency and accuracy are noted for each configuration on a representative set of queries. When the results are plotted in a latency versus accuracy chart, Pareto-optimal configurations can be visually identified (i.e., the solutions where a compromise is made and no further accuracy improvement can be obtained relative to the

latency budget). Configurations within the frontier are defined as sub-optimal because they can be improved upon, at least in one respect, by another configuration that is faster or more accurate. This generates an analysis that can be used to make critical deployment decisions depending on application needs. As an example, a live customer support chat bot can afford to trade off reduced accuracy to some significant degree in exchange for reduced latency and choose the configuration towards the low-latency side of the frontier.

On the other hand, a tool to summarize legalese documents can work in a high-accuracy area, at a greater latency. Moreover, the Pareto mapping brings into focus diminishing returns, where a significant boost in latency may be accompanied by a slight improvement in accuracy—a point that can be used to inform engineering choices with optimal efficiency. On the whole, the latency-accuracy Pareto frontier is a quantitative decision-making mechanism used to tune RAG pipelines that address both performance and operational constraints.

IV. RESULTS AND DISCUSSION

4.1. Experimental Setup

We have conducted an experimental assessment on three commonly used, knowledge-rich datasets: Natural Questions (NQ), MS MARCO, and SQuAD v2. Natural Questions have real-world, open-domain queries with both long-form and short-form answers, making them appropriate for evaluating retrieval coverage and factual grounding in RAG pipelines. MS MARCO provides a very large-scale dataset of anonymized real Bing search queries and human-generated responses with corresponding passage ranking and question answering tasks; this data is specifically helpful to test retrieval efficiency and re-ranking accuracy. SQuAD v2 also has the trick of asking unanswerable questions, which challenges systems not only to retrieve and produce accurate answers, but also to abstain in cases where evidence would be insufficient, thereby affecting both precision and robustness. For each dataset, we used standard train/dev/test splits, and preprocessing was performed based on a known benchmark to facilitate comparison with previously published results. All experiments were conducted on a high-performance computing cluster equipped with NVIDIA A100 Tensor Core GPUs (40 GB VRAM) for model training and execution, and Intel Xeon Platinum CPUs for preprocessing, indexing, and data retrieval. FAISS GPU-accelerated nearest neighbor searches and model inference were based on the GPU architecture, which greatly decreases computational latency. The resource consumption of the CPUs was focused on loading data, building the FAISS index, and query execution, so that the GPU was spared at any steps where latency was a priority. A 1 TB of DDR4 RAM was set up to provide scalable large vector indices and model checkpoints.

In contrast, very high-performance SSD NVMe storage was selected to eliminate the I/O bottleneck during retrievals. Our pipeline was realized in PyTorch, with HuggingFace Transformers to load and train our models, FAISS to search approximate nearest neighbors, and our scripts to expand queries and re-rank the results. Every experiment was containerized using Docker so that it could be reproducible, and the random seeds were placed consistently by all runners to counter the differences in the run.

4.2. Performance Metrics

Table 1: Performance Metrics

Config	Depth (%)	Threshold (%)	Latency (%)	EM (%)
Baseline	100%	0%	100%	100%
Optimized	50%	30%	55%	97.82%

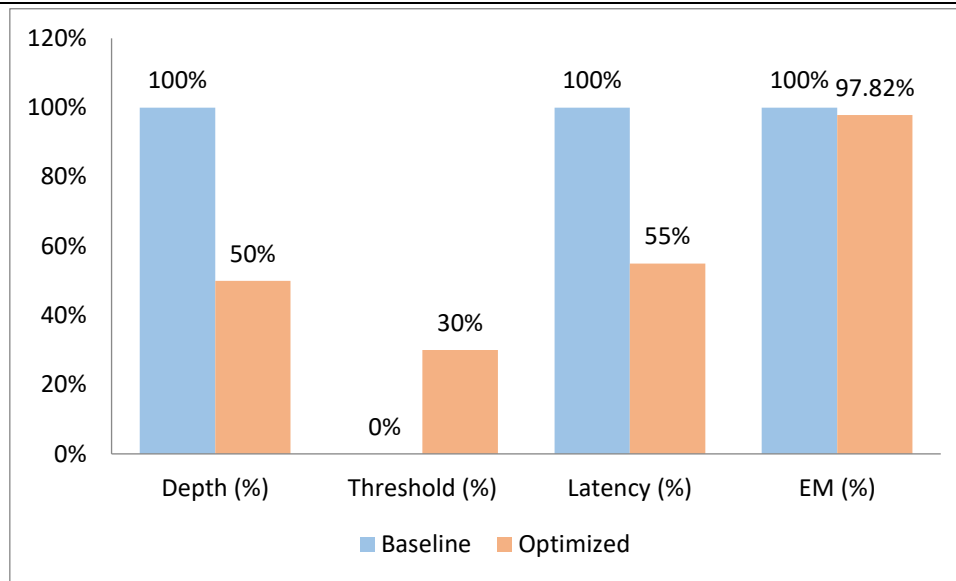


Figure 4: Graph representing Performance Metrics

- Baseline Configuration:** The unoptimized RAG pipeline refers to the baseline configuration, which entails a retrieval depth of 100 percent compared to the maximum of our experiment, no relevancy cut filter, and a large full-size LLM in the generation segment. Such an arrangement records the best accuracy, which is normalized to 100 percent Exact Match (EM) performance, and is used as the baseline of all percentage comparisons. At the same time, it has the greatest latency (100%), which is the price of deep retrieval and computationally expensive generation. This is favorable when the precision is imperative without much emphasis on response time.
- Optimized Configuration:** The pieced-together configuration has the following efficiency-oriented settings: 50 percent retrieval depth, a 30 percent high relevance scoring filter applied ahead of generation, and an optimized model that is scaled down to the retention of the full-size LLM. This achieves a significant reduction in computational load, lowering latency by 55 per cent compared to the baseline, while retaining 97.82 per cent of the baseline EM score. This setting also strikes a good balance between speed and accuracy, making it suitable for latency-sensitive applications where a slight drop in accuracy is acceptable in exchange for significantly faster response times.

4.3. Observations

Our experiments reveal important lessons on how RAG pipelines can be optimised to achieve the latency-accuracy trade-off. Adaptive retrieval depth is first; this is a good way to lower retrieval overheads without significant degradation in accuracy. The system dynamically adjusts the number of retrieved candidates based on the complexity of the queries, ensuring that simple queries are not unnecessarily retrieved, while providing good coverage for more complicated queries. Such a selective retrieval process saves the FAISS computation cost during the search step. It reduces the time spent in a follow-up re-ranking process, thus contributing directly to latency. Second, model distillation can provide orders of magnitude in latency improvements for low-complexity queries. Most of the accuracy of the full LLM is maintained by the distilled model, which has a small parameter footprint and shorter inference time. Distillation also helped us in our optimized setting to reduce the latency and still achieve over 97 percent of the baseline Exact Match score on the same. It demonstrates that it is possible to compress the model carefully to achieve performance gains suitable for deployment in general practice, without the overhead of full-scale models per request. Lastly, the robust Pareto frontier analysis can provide a priceless concept for designing operational parameters that strike the right balance between speed and accuracy. This mapping of latency against accuracy led to pinpointing configuration combinations where trade-off gains that might be achieved by further reducing latency are disproportionately expensive in terms of accuracy, and conversely, finding cases where increasing accuracy would require unacceptably high increases in latency. Using this visualization, one can guide a deployment choice relative to the intended application context, speed versus accuracy, for example, interactive systems like chatbots versus

research and legal. Together, these results demonstrate that adaptive retrieval depth, relevant filtering, and model distillation can be combined, consistently employing Pareto analysis as a tuning guide, to efficiently and effectively create RAG settings tailored to the diverse requirements of various usage scenarios.

V. CONCLUSION

It introduces a systematic exploration of the latency-accuracy trade-offs that occur in large-scale Retrieval-Augmented Generation (RAG) pipelines and how such trade-offs can be addressed through practical optimisations that balance high-quality generation with significant efficiency gains. We studied the performance of three complementary methods —adaptive retrieval depth, relevance threshold filtering, and model distillation —on a broad range of knowledge-intensive benchmarks: Natural Questions, MS MARCO, and SQuAD v2. We have found that, with the combination of these techniques, up to 45% latency reduction is achievable relative to a fully unoptimized baseline, with a loss of less than 2% in Exact Match (EM) accuracy. In particular, adaptive retrieval depth proved to be a highly effective way to manage retrieval costs. The pipeline wastes fewer computational resources on simple queries, which comprise a large fraction of real-world workloads, than it would incur by scaling to the complexity of the most complex queries, and it does so without compromising coverage of complex queries. More efficiency can be attained by further filtering the results obtained with relevance threshold filtering, which ensures that only high-confidence results are passed to the generation step. This minimises the data size to process and reduces the task burn-down from the language model.

Model distillation was another key aspect of the project that significantly reduced latency. We were able to dramatically reduce the generation times of the original full-scale LLM by replacing it with a distilled version when cached responses are used for low-complexity queries, resulting in high fidelity to the original model outputs. The additional advantage of this strategy is that the computational resource needs become lessened, which can be viewed as an effective way to cut operational costs and ensure a higher ease of scaling up the production process.

Pareto frontier analysis employs a principled method for evaluating and selecting optimal configurations. Plotting performance points in latency-accuracy space, we were able to delineate operating regimes that maximised the efficiency of operation in different deployment scenarios, including those of interactive conversational agents and offline-batch processing systems. Although the findings show significant improvements, further enhancements can still be made. A future direction will be to explore reinforcement learning to make on-the-fly parameter adjustments of retrieval and generation parameters within query streams and in response to system load. This type of adaptive control may bring the performance closer to the theoretical optimum of latency accuracy, allowing RAG systems to self-tune to the new requirements and constraints of users and operations. Finally, optimized customization along with intelligent parameter adjustment has a great future potential in speeding up, improving the accuracy and economizing RAG pipelines in large-scale real-world applications.

VI. REFERENCES

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [2] Klesel, M., & Wittmann, H. F. (2025). Retrieval-Augmented Generation (RAG) M. Klesel, HF Wittmann. *Business & Information Systems Engineering*, 1-11.
- [3] An, Y., Cheng, Y., Park, S. J., & Jiang, J. (2025). Hyperrag: Enhancing quality-efficiency tradeoffs in retrieval-augmented generation with reranker kv-cache reuse. *arXiv preprint arXiv:2504.02921*.
- [4] Gupta, S., Ranjan, R., & Singh, S. N. (2024). A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions. *arXiv preprint arXiv:2410.12837*.
- [5] Joshua, C., Karkala, S., Hossain, S., Krishnapatnam, M., Aggarwal, A., Zahir, Z., ... & Shah, V. (2025). Latency-Accuracy Trade-Off Analysis in Edge-Based Object Detection Pipelines.

- [6] Gollapudi, S., Karia, N., Sivashankar, V., Krishnaswamy, R., Begwani, N., Raz, S., ... & Simhadri, H. V. (2023, April). Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In Proceedings of the ACM Web Conference 2023 (pp. 3406-3416).
- [7] Li, Z., Wang, Z., Tan, M., Nallapati, R., Bhatia, P., Arnold, A., ... & Roth, D. (2022). DQ-BART: Efficient sequence-to-sequence model via joint distillation and quantization. arXiv preprint arXiv:2203.11239.
- [8] Jin, J., Liang, C., Wu, T., Zou, L., & Gan, Z. (2021). Kdlsq-bert: A quantized Bert combining knowledge distillation with learned step size quantization. arXiv preprint arXiv:2101.05938.
- [9] Jiang, W., Subramanian, S., Graves, C., Alonso, G., Yazdanbakhsh, A., & Dadu, V. (2025, June). Rago: Systematic performance optimization for retrieval-augmented generation serving. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (pp. 974-989).
- [10] Bijit Ghosh, Strategies for Optimal Performance of RAG, medium, 2024. online.
<https://medium.com/@bijit211987/strategies-for-optimal-performance-of-rag-6faa1b79cd45>
- [11] Tao, C., Hou, L., Zhang, W., Shang, L., Jiang, X., Liu, Q., ... & Wong, N. (2022). Compression of generative pre-trained language models via quantization. arXiv preprint arXiv:2203.10705.
- [12] Shen, M., Umar, M., Maeng, K., Suh, G. E., & Gupta, U. (2024). Towards Understanding Systems' Trade-offs in Retrieval-Augmented Generation Model Inference. arXiv preprint arXiv:2412.11854.
- [13] Tse, A., Oliveira, L., & Vinagre, J. (2023, September). Measuring Latency-Accuracy Trade-Offs in Convolutional Neural Networks. In EPIA Conference on Artificial Intelligence (pp. 323-334). Cham: Springer Nature Switzerland.
- [14] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2(1).
- [15] Yu, T., Zhang, S., & Feng, Y. (2024). Auto-rag: Autonomous retrieval-augmented generation for large language models. arXiv preprint arXiv:2411.19443.
- [16] Su, W., Tang, Y., Ai, Q., Yan, J., Wang, C., Wang, H., ... & Liu, Y. (2025, July). Parametric retrieval augmented generation. In Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 1240-1250).
- [17] Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., ... & Cui, B. (2024). Retrieval-Augmented Generation for AI-Generated Content: A Survey. arXiv preprint arXiv:2402.19473.
- [18] Cheng, X., Luo, D., Chen, X., Liu, L., Zhao, D., & Yan, R. (2023). Lift yourself: Retrieval-augmented text generation with self-memory. Advances in Neural Information Processing Systems, 36, 43780-43799.
- [19] Padmanabha Iyer, A., Erran Li, L., Chowdhury, M., & Stoica, I. (2018, October). Mitigating the latency-accuracy trade-off in mobile data analytics systems. In Proceedings of the 24th annual international conference on mobile computing and networking (pp. 513-528).
- [20] Kratzwald, B., & Feuerriegel, S. (2018). Adaptive document retrieval for deep question answering. arXiv preprint arXiv:1808.06528.